

Organic Bitcoin (OBTC) Whitepaper V1 (Implementation-Aligned, Extended Readable Edition)

Version: 1.0

Status: Request for Comments (RFC)

Audience: protocol engineers, wallet developers, mining pools, exchanges, researchers, operations teams, and community builders

0. Reading guide

The goal of this whitepaper is to **make the system clear**, not to bury the reader in jargon.

To lower the reading barrier, the suggested order is:

- **3-minute version:** Section 1, Section 2, Section 5.1
- **Product and operations version:** Section 6 (lifecycle), Section 11 (wallet), Section 14 (operations)
- **Engineering version:** Section 7 (index), Section 8 (REAP), Section 9 (replay protection), Section 13 (security)
- **Economics and governance version:** Section 12 (economics), Section 15 (governance)

This document uses English technical terms directly, such as UTXO, Replay Protection, Mempool, and Coinbase.

Each of them is also explained in plain language in **Section 19 (Glossary)**.

1. OBTC in one sentence

OBTC is a chain evolved from a Bitcoin fork:

- coins still exist as **UTXOs**;
- but every UTXO has a lifecycle;
- after expiry it can be reclaimed by a system-level mechanism called **REAP**:
 - **70%** is returned to the original script as a refund,
 - **30%** enters the miner security budget as tax;

- and **Namespace Isolation + Replay Protection** are used together to reduce cross-chain replay risk and accidental cross-chain transfers.

From a user point of view, the rule is:

Your asset is not "permanently static and automatically safe." It is an asset that requires periodic maintenance while remaining continuously protected by system rules.

1.1 Why it is called Organic

"Organic Bitcoin" does not mean "organic food Bitcoin."

It means **a Bitcoin system with metabolism.**

In comparison:

- Bitcoin behaves more like an inorganic system: once a UTXO is created it can remain still forever; state accumulates; and long-term miner security remains tied to the halving clock.
- OBTC behaves more like an organic system: assets go through creation, use, dormancy, expiry, reclaim, and regeneration; state can clean itself through REAP; and miner income gains an endogenous source from dormant supply returning to circulation.

So "organic" is not a branding word. It points to **lifecycle, recycling, and the system's ability to sustain itself.**

1.2 Where the innovation sits

Many of the underlying primitives in OBTC are not brand-new inventions, for example:

- UTXO expiry windows
- deterministic candidate selection and transaction construction
- OP_RETURN markers and commitments
- state summaries and index maintenance

The innovation is not mainly "new cryptography."

It is the combination of known mechanisms into an **executable economic system** that directly answers a question long ignored:

If permanent on-chain state is not free, who keeps paying for it over time?

2. Why build this at all

2.1 The real-world problem

The traditional UTXO model has a long-term structural issue:

- part of the supply will become permanently dormant because of lost keys, inaccessible estates, broken backups, or operational failures;
- those assets remain nominally on-chain while no longer participating in economic circulation;
- and the network security budget becomes increasingly uncertain as subsidy declines and fees become more important.

2.2 OBTC's answer

OBTC does not assume that permanent dormancy is the best default state.

Instead, it introduces an "organic metabolism" model:

1. the **expiry mechanism** brings long-unmoved assets into a governable state;
2. **system reclaim** routes part of dormant supply back into the security budget;
3. the **refund mechanism** avoids pure confiscation;
4. **wallet-side renewal tools** help active holders keep assets alive at much lower cost.

2.3 Design targets

- sustained security budget,
- clear asset lifecycle,
- deterministic behavior,
- operationally manageable systems,
- script neutrality after expiry.

2.4 Non-goals

- an asset model that lets users ignore maintenance forever,
 - permanent storage of all historical transaction bodies by every node,
 - complex treasury governance or discretionary distribution logic.
-

3. System overview: a three-layer architecture

OBTC can be understood as three layers with different responsibilities.

3.1 Consensus Layer

This layer answers: **what is a valid transaction and a valid block.**

Core responsibilities:

- expiry determination,
- REAP validity rules,
- replay-protection enforcement after activation.

3.2 Mining and Policy Layer

This layer answers: **within the valid set, how should blocks be assembled safely.**

Core responsibilities:

- REAP candidate selection and construction,
- reserving weight budget for REAP in block templates,
- separating system transactions from ordinary mempool policy.

3.3 Wallet Execution Layer

This layer answers: **how users discover risk and take action.**

Core capabilities:

- `obtc.getexpiry` : inspect expiry risk,
- `obtc.renew` : renew manually,
- `renewall` : renew in batches,
- Auto-Renew: automated renewal with failure backoff and budget ceilings.

4. The core mechanism in plain language

Start with the most intuitive example.

Assume you hold a `1 BTC` UTXO:

1. it is created at height `h_create` ;
2. it becomes expired at height `h_create + E` , where `E` is the expiry window;
3. if you renew before that point, the asset remains under your active control;
4. if you never do anything, the system may execute REAP inside a block:
 - return `0.7 BTC` to the original script,
 - count `0.3 BTC` as miner tax,
 - start a new timer for the refund output.

The key points are:

- it does **not** drop to zero;
- it is **not** handled arbitrarily;
- it is a rule-based, verifiable, predictable on-chain behavior.

5. Parameters and activation schedule (current implementation values)

5.1 Activation matrix across the three networks

Network	Fork Height	Expiry Index Start	Expiry Enable	REAP Hardening	Replay Protection
Mainnet	950000	950000	1050000	1060000	1065000
Testnet	2800000	2800000	2800100	2800120	2800130
Regtest	100	100	110	112	114

Explanation:

- **Fork Height:** the height where OBTC enters its own rule domain.
- **Expiry Enable:** the point at which expiry spending constraints become active.
- **REAP Hardening:** the point where REAP input order and input caps become consensus-enforced.
- **Replay Protection:** the point where signature-domain isolation becomes consensus-enforced.

5.2 Expiry parameters

Network	WindowBlocks	ListBatchLimit	ReapMaxInputs
Mainnet	362,880	10,000	256
Testnet	1,008	5,000	500
Regtest	144	1,000	200

Notes:

- Mainnet 362,880 blocks equals 9! and is about 6.91 years;
- test networks use shorter windows to make validation and rehearsal faster.

5.3 Namespace Isolation parameters

- Network magic:
 - Main: 0x4F425443
 - Test: 0x4F544553
 - Reg: 0x4F524547
- Default port: 9527 / 19527 / 29527
- Bech32 HRP: obtc / obtct / obtcrt
- isolated address prefixes for P2PKH, P2SH, and witness formats
- isolated HD key versions
- BIP44 coin type: 20260 / 20261 / 20262

Implementation requirement: collisions are checked at startup, and the node must refuse to start if a conflict is detected.

5.4 Default REAP policy parameters

- Sort mode: Strict
- Mainnet max inputs: 256
- Mainnet weight budget: 200,000
- Tax ratio: 30%
- Dust threshold: 720 sat (6! = 720)

5.5 Default Auto-Renew parameters

- Enabled: `false`
 - Interval: `30m`
 - Failure backoff: `15m` (may be `0`)
 - Window: `window_end <= blocks_to_expiry <= window_start`
 - `window_start = 52560`
 - `window_end = 25920`
 - MaxUtxosPerRun: `100`
 - MaxFeeRate: `5000 sat/KB`
 - MaxRenewAmountPerRun: `0` (unlimited)
 - `autorenewamount` : when enabled, it must be greater than `0`
-

6. UTXO lifecycle

6.1 The key formula

Let:

- `W = WindowBlocks`
- on mainnet `W = 362,880` blocks (`9!` , about `6.91` years; often communicated externally as "about 7 years")

For any UTXO:

$$h_expiry = h_create + W$$

When `h_tip >= h_expiry` , that UTXO becomes expired.

6.2 Wallet-side status layers

To make the system legible, wallets usually present three states:

- `ok` : still far from expiry
- `expiring` : approaching expiry and should be handled
- `expired` : already expired

Note: `ok` and `expiring` are UX layers; the actual consensus question is simply whether the UTXO is expired.

6.3 The two paths in the lifecycle

1. **User path:** renew before expiry and keep the asset active.
 2. **System path:** after expiry, the asset can be processed by REAP rules.
-

6.4 A full example

- UTXO amount: `10,000,000 sat` (`0.1 BTC`)
 - after expiry and REAP:
 - `tax = floor(10,000,000 x 30%) = 3,000,000 sat`
 - `refund = 7,000,000 sat`
 - if `refund` is below the dust threshold, no refund output is created and the full amount is folded into tax.
-

7. Expiry Index: how to track expiring assets efficiently

7.1 Why an index is mandatory

It is not acceptable to scan the entire historical set on every new block just to ask "what expired now?"

That is why a dedicated expiry index is required.

7.2 Bidirectional mapping structure

At an implementation level, the index can be abstracted as:

1. `OutPoint -> ExpiryKey`
2. `ExpiryKey -> OutPoint set`

Benefits:

- fast deletion when an output is spent,
 - fast range scanning by walking `ExpiryKey` in order.
-

7.3 Block connection and rollback

- `ConnectBlock` : add new outputs into the index, remove spent inputs from the index;
- `DisconnectBlock` : restore the reverse state.

That keeps the index aligned with the active chain even under reorg.

7.4 Paged scanning and resumable queries

Suggested interface:

```
(fromKey, toKey, maxResults, startAfter)
```

- `maxResults` controls page size,
- `startAfter` supports resumable scans,
- suitable for large queries without returning excessive result sets in one response.

8. REAP: the full rule set for system reclaim transactions

8.1 What REAP is

REAP is a block-internal system transaction:

- its purpose is to process expired UTXOs;
- it is not an ordinary user transaction template and does not rely on mempool relay.

8.2 Minimum conditions for validity

For a REAP transaction to be valid, at least the following must hold:

- all inputs must be expired UTXOs;
- the transaction structure must match REAP characteristics;
- marker consistency checks must pass;
- after activation, hardening constraints must also hold.

8.3 Tax, refund, and dust rules

For each input amount `v` :

- `tax = floor(v x 30 / 100)`
- `refund = v - tax`

Dust fold rule: if $0 < \text{refund} < \text{dust_threshold}$, then:

- $\text{refund} \rightarrow 0$
- $\text{tax} += \text{refund}$

8.4 Marker binding

The marker format is:

$\text{REAP}:\langle\text{height}\rangle:\langle\text{count}\rangle:\langle\text{digest}\rangle$

Consensus checks:

- height must match,
- count must match,
- digest must match.

8.5 Hardening after activation

REAP inputs must follow canonical order:

$\text{expiry} \rightarrow \text{amount} \rightarrow \text{outpoint}$

and also respect the input ceiling ReapMaxInputs .

8.6 Coinbase accounting

Define:

- $\text{ReapTax}(h)$: the total REAP tax collected in the block at height h

The coinbase upper bound is:

$\text{BaseSubsidy}(h) + \text{Fees}(h) + \text{ReapTax}(h)$

ReapTax follows the same maturity rule as coinbase.

8.7 Invariant

The following must always hold:

$\text{sum}(\text{inputs}) = \text{sum}(\text{refunds}) + \text{sum}(\text{tax})$

This is the core audit and regression invariant.

9. Replay Protection: how cross-chain replay is prevented

9.1 Two defensive layers

1. **Namespace Isolation** at the address, port, HD, and coin-type level.
2. **Replay-protected sighash domain** at the signature-message level.

9.2 Implementation details

- replay bit: `0x40`
- domain tags: separate tags for Legacy, Witness, and Taproot paths

9.3 Activation gate

- before activation: compatibility semantics may continue to work;
- after activation: any signature missing replay-protected semantics fails directly.

9.4 Path coverage

- Legacy
- SegWit v0
- Taproot, both key path and script path

10. Node operation model

10.1 Pruned Full Node vs Archive Node

- **Pruned Full Node**: keeps the full UTXO set, a rolling window of block bodies, and all block headers.
- **Archive Node**: keeps the full historical chain and all transaction bodies.

10.2 Invariants

- history is never rewritten;
- pruning changes storage retention, not consensus verifiability.

10.3 Snapshot synchronization and UTXO commitments

Initial sync can be accelerated through snapshot import plus proofs plus multi-source consistency checks.

At least k independent sources should agree before an import is trusted.

11. Wallet capability closure

11.1 `obtc.getexpiry`

Returns the key risk fields for expiry management:

- outpoint
 - amount
 - create height
 - expiry height
 - blocks_to_expiry / days_to_expiry
 - status
 - dust_risk
-

11.2 `obtc.renew`

Supports explicit renewal by outpoint, with parameters including:

- amount
- target address (optional)
- max fee rate (optional)
- minconf (optional)

It returns a transaction summary such as txid, number of inputs and outputs, and fee rate.

11.3 `renewall`

Supports:

- status or window-based filtering,
 - dry-run mode,
 - interval/runs scheduling.
-

11.4 Auto-Renew scheduler

Features:

- periodic execution,
- window-based filtering,
- candidate count limit,
- fee-rate ceiling,
- failure backoff,
- per-run budget control.

The goal is automated risk mitigation within explicit safety boundaries.

11.5 The wallet direction for AI agents

OBTC has a natural extension path at the wallet layer: **make lifecycle asset management suitable for machine actors, not only for human operators.**

The reason is straightforward:

- agents cannot independently complete KYC or open traditional bank accounts;
- but they will increasingly pay for compute, buy data, execute budgets, and manage treasuries;
- and the core actions in OBTC are continuous observation, rule-based renewal, batch scheduling, and auditable execution, all of which align well with machine workflows.

That is why the interface and wallet layer should prioritize:

- capability-based authorization instead of all-or-nothing unlock semantics,
- a controlled pipeline of `preview -> approve -> sign -> publish` ,
- separation between a watch-only planner and an isolated signer,
- event-driven alerts for expiry, failure, and policy conditions,
- default auditability through operation metadata and policy metadata.

This is a **wallet and product-layer direction**, not a new consensus rule.

12. Economics and game theory

12.1 Long-term security budget estimate

Let mainnet $W = 362,880$ blocks. Using $52,560$ blocks per year:

$$T = W/52,560 \approx 6.91 \text{ years}$$

For a post-REAP retention ratio $\rho = 0.7$:

$$p = -\ln(\rho)/T \approx 5.2$$

If the dormant-supply ratio is L :

$$B \approx L \cdot p$$

When $L \in [20\%, 30\%]$, then B is roughly $1.0\% - 1.6\%$ per year.

12.2 Incentives for compliant users

When the cost ratio of renewal ϕ is far below the 30% tax rate, rational users prefer proactive renewal.

That means larger UTXOs will usually be actively maintained, while smaller UTXOs are more constrained by dust economics.

12.3 MEV suppression

Deterministic ordering plus resource ceilings compress the room for preferential harvesting and reduce strategic noise.

12.4 Immediate expired backlog from historical inheritance

OBTC does not start from an empty genesis. It inherits Bitcoin's historical UTXO set.

Let:

- $F = \text{Fork Height}$
- $A = \text{Expiry Enable}$
- $W = \text{WindowBlocks}$

Then every historical UTXO satisfying:

$$h_{\text{create}} + W \leq A$$

and still unspent at activation immediately enters the expired backlog once expiry is enabled.

Using the current mainnet parameters:

- $F = 950,000$
- $A = 1,050,000$
- $W = 362,880$
- therefore the threshold is $h_{create} \leq 687,120$

This means the system does **not** have to wait a full 6.91 years before the first large historical dormant set becomes processable.

Once the activation height is reached, the chain already contains a pre-existing candidate pool.

Important nuance:

- the backlog exists immediately,
- but it is released gradually, block by block, because each REAP transaction is still constrained by `ReapMaxInputs` and the weight budget,
- so it behaves more like a long inventory processed over many blocks than a one-block purge.

12.5 The economic flywheel: why it may self-start

The potential flywheel in OBTC is not a single narrative; it is a linked set of reinforcing loops:

1. historical dormant outputs create an immediate backlog;
2. REAP gradually turns that backlog into tax flow;
3. tax flow strengthens miner revenue and security-budget resilience;
4. a more stable security expectation improves willingness to integrate among miners, wallets, and platforms;
5. more integration increases renewal, transfer, and asset-management activity on-chain.

If AI-agent adoption is layered on top, the flywheel gains another source of real demand:

- agents need permissionless, machine-callable, authorizable, auditable settlement assets;
- the OBTC lifecycle model treats continuous monitoring, policy execution, and auditable risk handling as first-class behaviors.

13. Security model and threat analysis

13.1 Typical threats

- forged markers,
 - reordered REAP inputs,
 - oversized input sets,
 - cross-chain replay,
 - fake system transactions injected toward the mempool,
 - poisoned snapshots,
 - reorg-driven arbitrage.
-

13.2 Defensive lines

- marker consistency checks,
 - canonical order,
 - input caps plus weight budgets,
 - replay-protected sighash domains,
 - mempool policy isolation,
 - multi-source snapshot validation,
 - coinbase maturity constraints.
-

14. Operations and observability

14.1 Suggested metrics

Consensus

- expired spend rejection count,
- REAP non-expired spend rejection count,
- replay-protection violation count.

Mining

- template build attempts with REAP,
- REAP append success rate,
- reserved weight utilization,
- REAP tax contributed to coinbase.

Wallet

- auto-renew candidate count per run,
 - auto-renew success/failure ratio,
 - backoff activation count,
 - per-run budget truncation count.
-

14.2 Suggested alerts

- N consecutive rounds of auto-renew failure,
 - sustained REAP append success below threshold,
 - expiring backlog continuing to rise.
-

14.3 Implementation suggestions

1. consensus layer: end-to-end tests for Expiry, REAP, and Replay Protection
 2. node layer: regression for both pruning and archival modes
 3. wallet layer: expiry warnings, manual renewal, and auto-renew stability tests
 4. mining layer: block-template boundary tests and stress tests
 5. test networks: accelerated-expiry rehearsals, for example around seven days
 6. tooling layer: public dashboards and anomaly-tracking reports
-

15. Governance and upgrades

Governance principles:

- core constants should remain as stable as possible,
 - adjustable parameters must have delayed effect and explicit activation heights,
 - upgrade flow should be mechanical, transparent, and reproducible,
 - discretionary space should be minimized.
-

16. Mathematical appendix

- mainnet time window: $T = W / 52,560$, where mainnet $W = 362,880$
- annualized decay: $p = -\ln(\rho) / T$
- annual security inflow: $B = L \times p$
- equal-budget transformation: $\rho' = \exp(-pT')$

- reverse solution for target budget: $\rho = \exp(-(B_{\text{target}} \times T)/L)$, $\tau = 1 - \rho$
-

17. Key KPIs

- `ReapTax` / `MinerRevenue` on an annualized basis,
 - supply at risk (expiring within 90 days),
 - renewal success rate,
 - average and quantile volatility of REAP counts per block,
 - number and distribution of archive nodes,
 - auto-renew adoption rate and complaint rate.
-

18. Legal and compliance notice

OBTC is an independent chain and is not the same as Bitcoin.

The expiry and reclaim mechanism may be interpreted in different jurisdictions as a protocol fee, a negative interest mechanism, or a redistribution rule applied to dormant assets.

Exchanges and custodians should design their internal processes and user disclosures accordingly.

19. Glossary

This section exists to lower the reading barrier. If a term feels unfamiliar, check here first.

Term	English	Plain-language meaning	Role in OBTC
Unspent output	UTXO	A spendable piece of value on-chain	The basic accounting unit in OBTC
Output locator	OutPoint	<code>txid:vout</code> , the unique coordinate of one output	Used to pinpoint a UTXO precisely
Expiry	Expiry	The point where an asset reaches its protocol age limit	Triggers renewal or system reclaim
Expiry height	Expiry Height	The block height where a UTXO becomes expired	The hard consensus threshold
Expiry index	Expiry Index	The data structure tracking who expires when	Makes scanning and deletion efficient
System reclaim	REAP	The protocol mechanism that processes expired UTXOs inside blocks	Returns part of dormant value into the security budget
REAP transaction	REAP Tx	The system transaction used to execute REAP	Not an ordinary user transaction template
Refund	Refund	The 70% portion returned to the original script	Preserves continuity of ownership
Tax	Tax	The 30% portion routed to miner revenue	Forms a continuing security-budget source
Marker output	Marker	The OP_RETURN string <code>REAP:height:count:digest</code>	Binds inputs and context together
Canonical order	Canonical Order	The unique agreed input ordering	Prevents divergence caused by implementation differences

Term	English	Plain-language meaning	Role in OBTC
Mempool	Mempool	The staging area for unconfirmed transactions	REAP is rejected here
Block template	Template	The candidate block a miner prepares	REAP is appended conditionally here
Coinbase transaction	Coinbase	The reward-settlement transaction in a block	ReapTax is reflected through its upper-bound rule
Replay protection	Replay Protection	Signature-domain rules that prevent cross-chain replay	Mandatory after activation
SigHash type	SigHash	Defines what part of a transaction the signature commits to	Carries the replay bit and domain semantics
Domain separation	Domain Separation	Different tags for different signature spaces	Prevents signature reuse across chains
Namespace isolation	Namespace Isolation	Isolation of addresses, ports, HD versions, and related identifiers	Prevents accidental cross-chain confusion
Dynamic dust	Dynamic Dust	A dust threshold that adapts to fee conditions	Helps avoid uneconomic outputs
Auto-Renew	Auto-Renew	Wallet-side automated renewal	Reduces the risk of user inaction
Failure backoff	Failure Backoff	Delay before the next retry after a failure	Prevents retry storms
Per-run budget	Per-Run Budget	Maximum renewal amount per scheduler round	Prevents abnormal overspending

Term	English	Plain-language meaning	Role in OBTC
Pruned full node	Pruned Full Node	A node that keeps only the necessary rolling window	Reduces storage cost
Archive node	Archive Node	A node that keeps full history	Supports full-history query and audit
Reorg	Reorg	A switch from one chain tip to another	Requires index and state rollback consistency
DoS	DoS	Resource exhaustion attacks	Mitigated by caps and budgets
Invariant	Invariant	A rule that must always hold	Anchor for correctness checking

20. Frequently asked questions

Q1: What happens if I do nothing?

If your UTXO remains unspent long enough to reach expiry, it enters the expired state. After that it is no longer spent through the ordinary path and may be processed by REAP: 70% refund and 30% tax to the security budget.

Q2: Does the asset get taken away immediately at the expiry point?

No. Expiry means the UTXO enters a state where the system is allowed to process it. The exact block where REAP happens depends on whether the block template selects that input in that round.

Q3: How do I avoid REAP?

Renew before expiry.

You can do this manually with `obtc.renew`, or use batch and automated renewal strategies.

Q4: Why is there a 30% tax?

This is part of the protocol-level security-budget design.

It routes part of dormant asset value into miner income and strengthens long-term budget resilience. For active users, renewal is usually much cheaper than 30%.

Q5: Are replay protection and address-prefix isolation redundant?

No. They work at different layers:

- address, port, HD, and related namespace isolation prevent accidental misconnection and misdirected transfers;
- signature-domain isolation prevents cross-chain signature reuse.

Both are required for a complete defense.

Q6: Why does REAP not enter the mempool?

Because it is a system transaction, not an ordinary user transaction.

Putting it into the mempool would invite fake-system-transaction pollution and resource abuse. The current model is block-template construction plus in-block execution.

Q7: What happens to small UTXOs?

If the refund amount falls below the dust threshold, dust fold is triggered: no refund output is created, and the refund amount is merged into tax. That avoids creating on-chain garbage outputs.

Q8: I only care about engineering implementation. Which sections matter most?

Recommended order: Section 5 (parameters) -> Section 8 (REAP) -> Section 9 (Replay Protection) -> Section 11 (Wallet) -> Section 14 (Operations).

Q9: Does this rule set make the system too complex?

The complexity mainly comes from making long-term risk explicit.

OBTC's approach is to move that complexity into rules and tooling earlier, instead of leaving the system to rely on manual after-the-fact remedies.

Q10: If parameters need to change in the future, how do we avoid arbitrary changes?

Parameter upgrades should follow the principle of delayed effect plus fixed activation height plus a transparent signaling process. The main objective is to reduce governance uncertainty and market expectation shocks.

Q11: Why call it Organic Bitcoin?

Because it treats assets as part of a system with a lifecycle rather than a permanently static pile of state. UTXOs move through creation, use, dormancy, expiry, reclaim, and regeneration. "Organic" points to a living system, not a food metaphor.

Q12: Why does the whitepaper mention AI agents?

Because the wallet layer in OBTC naturally involves continuous expiry monitoring, batch renewal, policy execution, and audit traces. Those are all things machine actors are good at. This is a **wallet and product direction**, not an attempt to write AI into consensus.

21. Conclusion

The core of OBTC is not novelty for its own sake. It is the executability of the rule set.

Its distinctiveness does not mainly come from brand-new cryptographic primitives. It comes from combining known mechanisms into a working economic system.

OBTC puts four questions into one coherent architecture:

1. how long-dormant assets should be handled,
2. how miner security budgets can stay resilient over long time horizons,
3. how cross-layer systems can remain predictable, operable, and explainable,
4. how machine actors may eventually use assets safely within explicit boundaries of authorization, renewal, and audit.

The only real success criterion for this whitepaper is:

Are the rules clear enough that independent teams can implement them separately and still converge on the same chain behavior?

22. License

This document is recommended for publication under **CC BY 4.0**.

Reference implementations may use permissive licenses such as MIT.

End of Whitepaper V1.0 (Implementation-Aligned, Extended Readable Edition)